

Model Reduction for 1-D Convection-Diffusion Equation

Saadbin Khan¹ and Mansoor Ahmed²
Oklahoma State University, Stillwater, Oklahoma, 74074

A significant body of work has been done by combining model reduction techniques and the Finite Difference Methods (FEM). Over the years, successful endeavors using this combination in solving complex engineering problems led to newer doors to more practical applications. In this project, an efficient and reliable model order reduction is used to reduce a high dimensional system of the ordinary differential equations (ODEs), which is not desirable for certain applications in terms of performance, costs, and so on. In this study, we first solved the linear one dimensional convection diffusion heat equation using the finite element method based on the principle of Galerkin projection to find our optimal solution in a finite space in the L2 norm sense. The comparison of the result of reduced order model to the full order finite element solution is presented, and shows that our approximation captures reasonable proportion of the dynamics of the system with fewer number of bases.

I. PROBLEM DESCRIPTION

Convection-diffusion equation is one of the most significant partial differential equations (PDE) to represent a huge number of physical phenomena and engineering problems related to heat transfer and fluid transport. It has the general form:

$$\frac{\partial c}{\partial t} = \nabla \cdot (D \nabla c) - \nabla \cdot (\vec{v} c) + R$$

Over the years many mathematicians solved this equation in different methods and applied numerous techniques to make the solution computationally effective. For this project, we are considering a 1-D version of the convection-diffusion equation which governs the temperature $\omega(x, t)$ in a thin rod:

$$\omega_t = \omega_{xx} + a\omega_x + \omega + 1, \quad a < 0$$

here:

$$\begin{aligned} \omega_t &= \text{unsteady term (time-dependent)} \\ \omega_{xx} &= \text{diffusion term} \\ a\omega_x &= \text{convection (or advection) term} \\ \omega &= \text{variable source term} \\ 1 &= \text{constant forcing term} \end{aligned}$$

For this problem we consider $x \in [0,1]$ and $t \in [0,10]$ with the following boundary and initial conditions:

$$\begin{aligned} \text{Boundary conditions:} \quad & \omega(0, t) = 0 \text{ and } \omega(1, t) = 0 \quad \text{for } t > 0 \\ \text{Initial condition:} \quad & \omega(0, t) = \sin(\pi x) \end{aligned}$$

Our initial goal is to use finite element method (FEM) to discretize the PDE in space and solve for the full order system. Then we will use Proper Orthogonal Decomposition (POD) to reduce the order of the model to simulate the system performance faster without deteriorating the quality of the results significantly.

¹ Graduate Student, Mechanical and Aerospace Engineering (email: saadbin.khan@okstate.edu)

² Graduate Student, Mechanical and Aerospace Engineering (email: manso@ostatemail.okstate.edu)

II. METHODOLOGY

A. Full Order Solution

For the first part of the problem we will discretize the equation in space to apply finite element method. As the variable under study ω is both spatially and temporally dependent, so we consider the following to separate the variable:

$$\omega_h(x, t) = \sum_{i=1}^{N+1} \varphi_i(x) y_i(t)$$

Here, $\omega_h(x, t)$ is called the finite element solution for the equation we are dealing with and $\{\varphi_i\}_{i=1}^N$ is the finite element basis. The finite element snapshots $\{\omega_j\}_{j=1}^N$ at time $t_j, j = 1, 2, \dots, n$, are defined by

$$\omega_j = \omega_h(x, t_j) = \sum_{i=1}^{N+1} \varphi_i(x) y_i(t_j)$$

The foremost step of FEM is the weak formulation of the equation. We shall use the piecewise approximations to construct a finite element space U_h which is critical for the discretization of the weak formulation. U_h is an $(N+1)$ dimensional subspace of $C[a, b]$.

$$U_h = \text{span}\{\varphi_j\}_{j=1}^{N+1}$$

where φ_j is the basis function that is linear on each $[x_n, x_{n+1}]$ ($n = 1, 2, 3, \dots, N$). The definitions are:

$$\begin{aligned} \varphi_1(x) &= \begin{cases} \frac{x_2 - x}{h}, & \text{if } x_1 \leq x \leq x_2 \\ 0, & \text{otherwise} \end{cases} \\ \varphi_j(x) &= \begin{cases} \frac{x - x_{j-1}}{h}, & \text{if } x_{j-1} \leq x \leq x_j \\ \frac{x_{j+1} - x}{h}, & \text{if } x_j \leq x \leq x_{j+1} \\ 0, & \text{otherwise} \end{cases} \\ &\quad (i = 2, 3, \dots, N) \\ \varphi_{N+1}(x) &= \begin{cases} \frac{x - x_N}{h}, & \text{if } x_N \leq x \leq x_{N+1} \\ 0, & \text{otherwise} \end{cases} \end{aligned}$$

After defining finite element space, we introduce the weak formulation of our equation using a test function $v(x)$ as follows:

$$\begin{aligned} \int_0^1 \omega_t(x, t) v(x) dx &= \int_0^1 \omega_{xx}(x, t) v(x) dx + a \int_0^1 \omega_x(x, t) v(x) dx + \int_0^1 \omega(x, t) v(x) dx + \int_0^1 v(x) dx \\ \text{Or,} \quad \int_0^1 \omega_t(x, t) v(x) dx &= - \int_0^1 \omega_x(x, t) v_x(x) dx + a \int_0^1 \omega_x(x, t) v(x) dx + \int_0^1 \omega(x, t) v(x) dx + \int_0^1 v(x) dx \end{aligned}$$

Due to the mentioned boundary condition we require the test function $v(x) = 0$ at the boundaries $x = 0$ and $x = 1$. Now, using the finite element approximation of decoupled form $\omega_h(x, t) = \sum_{i=1}^{N+1} \varphi_i(x) y_i(t)$ and substituting our test function $v(x) = \varphi_j(x)$, we finally obtain the discretized system:

$$\sum_{i=1}^{N+1} y'_i \int_0^1 \varphi_i \varphi_j dx = \sum_{i=1}^{N+1} y_i \int_0^1 [-\varphi'_i \varphi'_j + a \varphi'_i \varphi_j + \varphi_i \varphi_j] dx + \int_0^1 \varphi_j dx$$

For $j = 0, 1, 2, \dots, (N + 1)$ which is basically a system of ODEs of the form:

$$M_h y'(t) = (-K_h + aF_h + N_h)y(t) + b$$

where,

$$y(t) = [y_1(t), y_2(t), y_3(t), \dots, y_{N+1}(t)]^T$$

$$M_h = \int_0^1 \varphi_i \varphi_j dx \quad F_h = \int_0^1 \varphi'_i \varphi_j dx \quad b = \int_0^1 \varphi_j dx$$

$$K_h = \int_0^1 \varphi'_i \varphi'_j dx \quad N_h = \int_0^1 \varphi_i \varphi_j dx$$

Here, the dimension of $y(t)$ and b is $(N+1) \times 1$ and the dimension of M_h, K_h, F_h, N_h is $(N+1) \times (N+1)$

Now, as the system is set up theoretically, we just need to find out the elements of all the matrices in the system of ODEs. We can use finite difference schemes to numerically find the integrations and differentiations or we can analytically calculate the diagonal elements of the matrices as most of the elements of the square matrices are zero rendering them tridiagonal in nature.

While solving this system of ODEs, we deliberately omitted the boundary elements of the matrices as we already know the value of $y(t)$ at boundaries. Our plan is to enforce the boundary values after solving the system containing the reduced square matrices of $(N-1) \times (N-1)$ and vectors $(N-1) \times 1$. One more advantage of using the piecewise polynomial approximation for spatial discretization is that we don't need to go through the extra trouble of calculating ω_h from the values found for y_i with the equation:

$$\omega_h(x, t) = \sum_{i=1}^{N+1} \varphi_i(x) y_i(t)$$

Because,

$$\varphi_j(x_k) = \delta_{jk} = \begin{cases} 0, & \text{if } j \neq k \\ 1, & \text{if } j = k \end{cases}$$

So,

$$\omega_h(x_k) = y_k$$

Using the handy ode45 function in MATLAB we solved the system and after enforcing the boundary values we found the temperature distribution over spatial and temporal domain which is our intended solution for full order.

B. POD-based Model Reduction

For this part, our goal is to approximate the subspaces of the ODE through finding a set of reduced number of orthogonal basis $\{\varphi_i\}_{i=1}^{K+1}$, with K being significantly lower than the full order basis number N , which minimizes the error in finite element approximation in a L^2 norm sense:

POD basis = Optimal solution of

$$\min_{\{\phi_i\}_{i=1}^{K+1}} \sum_{i=1}^{n+1} \|\omega_j - \tilde{\omega}_j\|_2^2 \quad \text{s.t.} \quad (\phi_i, \phi_j) = \delta_{ij}$$

where,

$$\tilde{\omega}_j(x) = \sum_{i=1}^{K+1} (\omega_j, \phi_i) \phi_i(x)$$

which is basically an approximation of ω_j using the POD basis $\{\phi_i\}_{i=1}^K$. Now we build a system of ODEs with reduced order. Let us consider that to be following:

$$\tilde{M}_h \tilde{y}'(t) = (-\tilde{K}_h + a\tilde{F}_h + \tilde{N}_h)\tilde{y}(t) + \tilde{b}$$

where,

$$\tilde{y}(t) = [\tilde{y}_1(t), \tilde{y}_2(t), \tilde{y}_3(t), \dots, \tilde{y}_{K+1}(t)]^T \in \mathbb{R}^{(K+1) \times 1}$$

similarly,

$$\tilde{M}_h, \tilde{K}_h, \tilde{F}_h, \tilde{N}_h \in \mathbb{R}^{(K+1) \times (K+1)} \text{ and } \tilde{b} \in \mathbb{R}^{(K+1) \times 1}$$

If we consider:

$$\tilde{\omega}_h^{K+1}(x, t) = \sum_{i=1}^{K+1} \phi_i^*(x) \tilde{y}_i(t)$$

Then, $\tilde{\omega}_h^{K+1}(x, t)$ is called the rank-K approximation of the finite element solution $\omega_h(x, t)$

To numerically computing the POD basis we did the following:

$$\mathcal{L} = [(\omega_i, \omega_j)] = \tilde{Y}^T M \tilde{Y} \in \mathbb{R}^{(n+1) \times (n+1)}$$

here,

$$M = [(\phi_i, \phi_j)] \in \mathbb{R}^{(N+1) \times (N+1)}$$

and

$$\tilde{Y} = \begin{pmatrix} y_1(t_1) & \cdots & y_1(t_{n+1}) \\ \vdots & \ddots & \vdots \\ y_{N+1}(t_1) & \cdots & y_{N+1}(t_{n+1}) \end{pmatrix}_{(N+1) \times (n+1)}$$

Then the POD basis is given by

$$\phi_i^* = \frac{1}{\sigma_i} \sum_{j=1}^{n+1} (v_i)_j \omega_j$$

Where, v_i satisfies the relation

$$\mathcal{L} v_i = \sigma_i^2 v_i, \quad i = 1, 2, \dots, K$$

Here v_i terms are sorted with respect to the descending order of σ_i . This sorting ensures that the error does not propagate and keeps to minimum. The error is given by:

$$\mathbf{e}_{K+1} = \sum_{i=1}^{K+1} \|(\omega_j, \phi_i^*) \phi_i^*(x)\|_2^2 = \sum_{i=K+1}^n \sigma_i^2$$

Now, the energy of the snapshots captured by first K+1 POD basis vectors are defined by:

$$\mathcal{E}_{POD}(K) = \frac{\sum_{i=K+2}^{n+1} \sigma_i^2}{\sum_{i=1}^{n+1} \sigma_i^2}$$

We need to choose the minimum integer for K such that

$$1 - \mathcal{E}_{POD}(K) \leq \epsilon \text{ for } \epsilon > 0$$

III. RESULTS

For this project we took $N = 20$ and thereby set $\Delta x = 0.05$ in the spatial domain. Similarly we set $\Delta t = 0.1$ to divide the considered time domain into 100 intervals. We considered three different values for 'a' which are -1, -10 and -100. Following are the 3-D plots for Temperature-time-length for each case. We can easily see from Figures 1 that, the decaying of the temperature with time is directly related to the magnitude of a. for a = 100, the Temperature decays the fastest. Also, Figure 2 gives the 2D plots for temporal distribution of Temperature at different points.

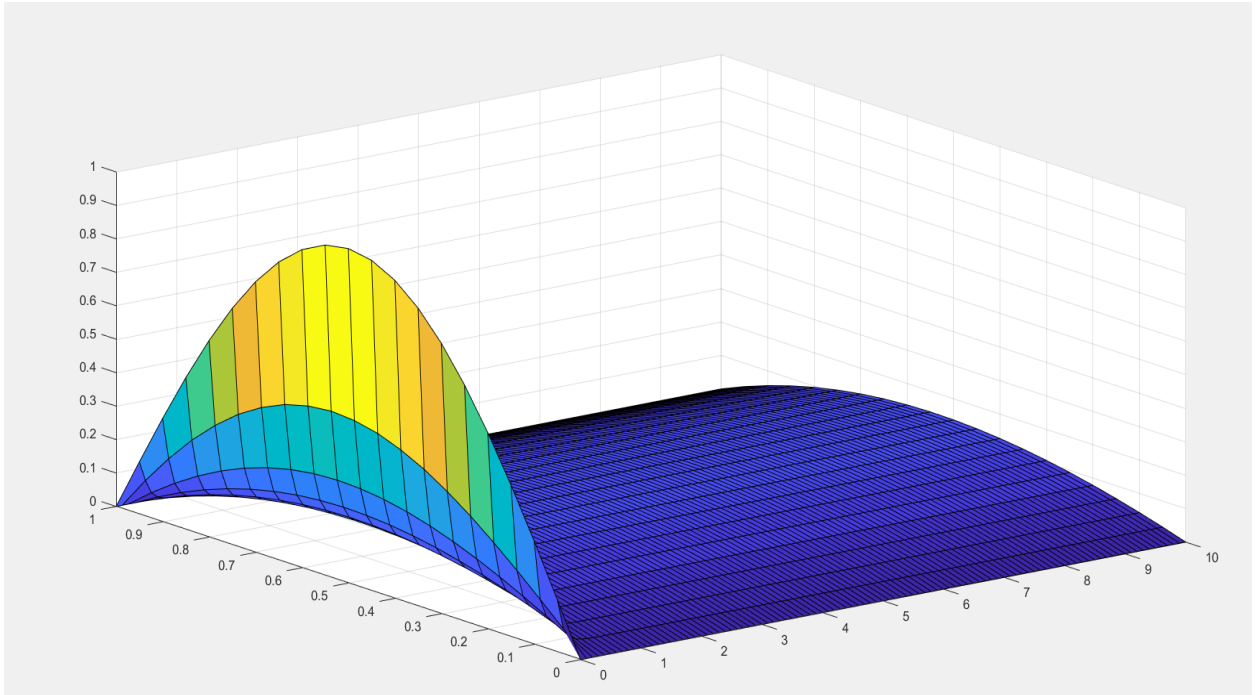


FIGURE 1(a): 3D temperature distribution for $a = -1$

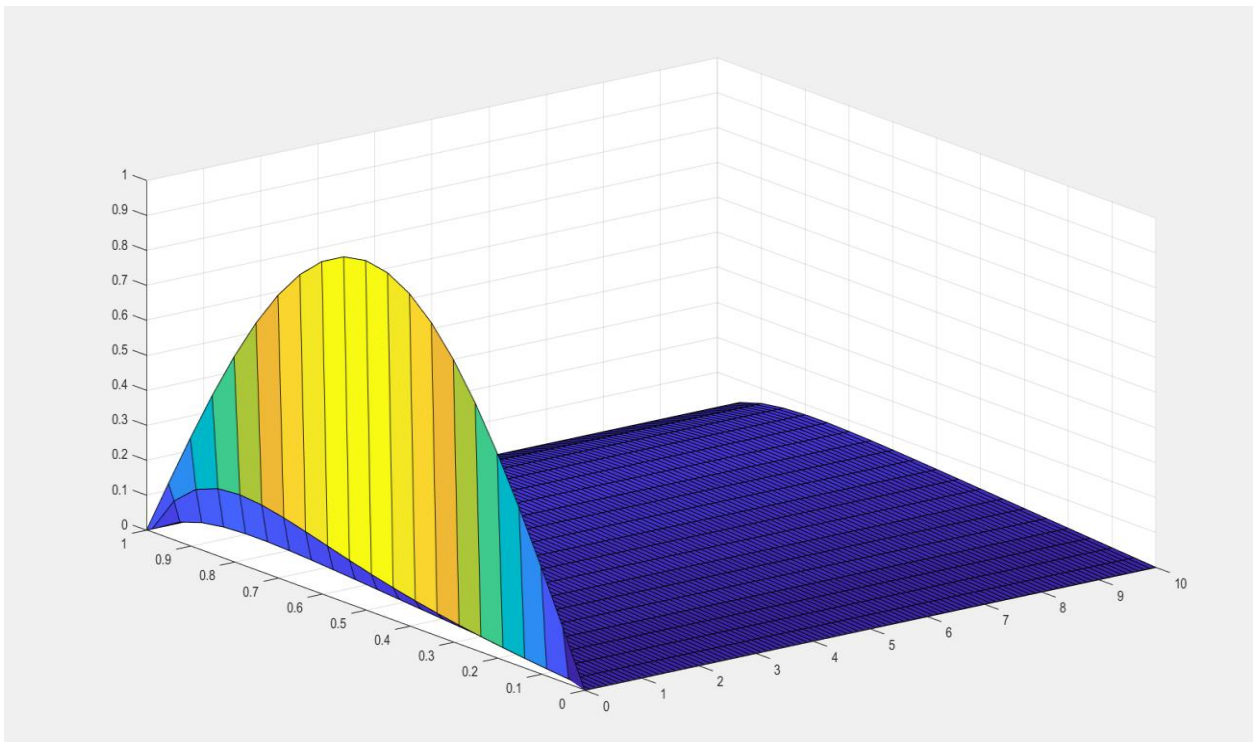


FIGURE 1(b): 3D temperature distribution for $a = -10$

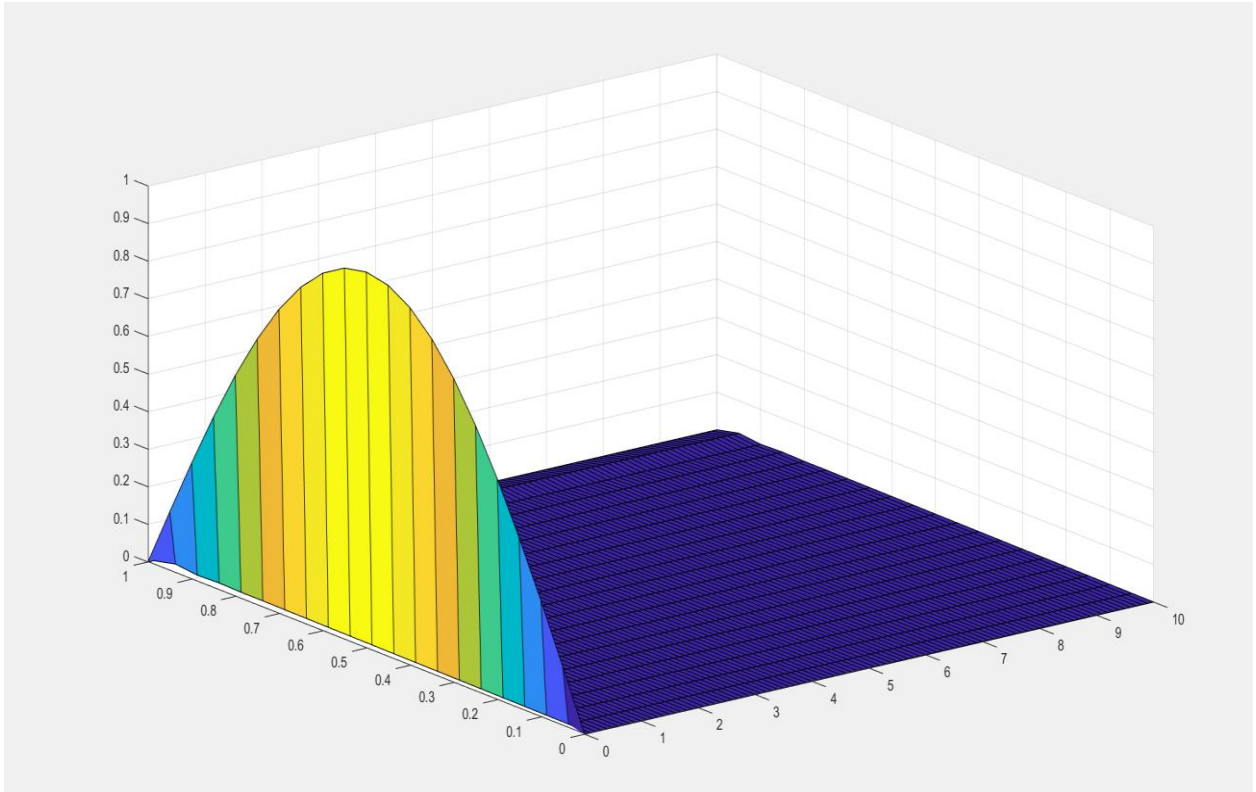


FIGURE 1(c): 3D temperature distribution for $a = -100$

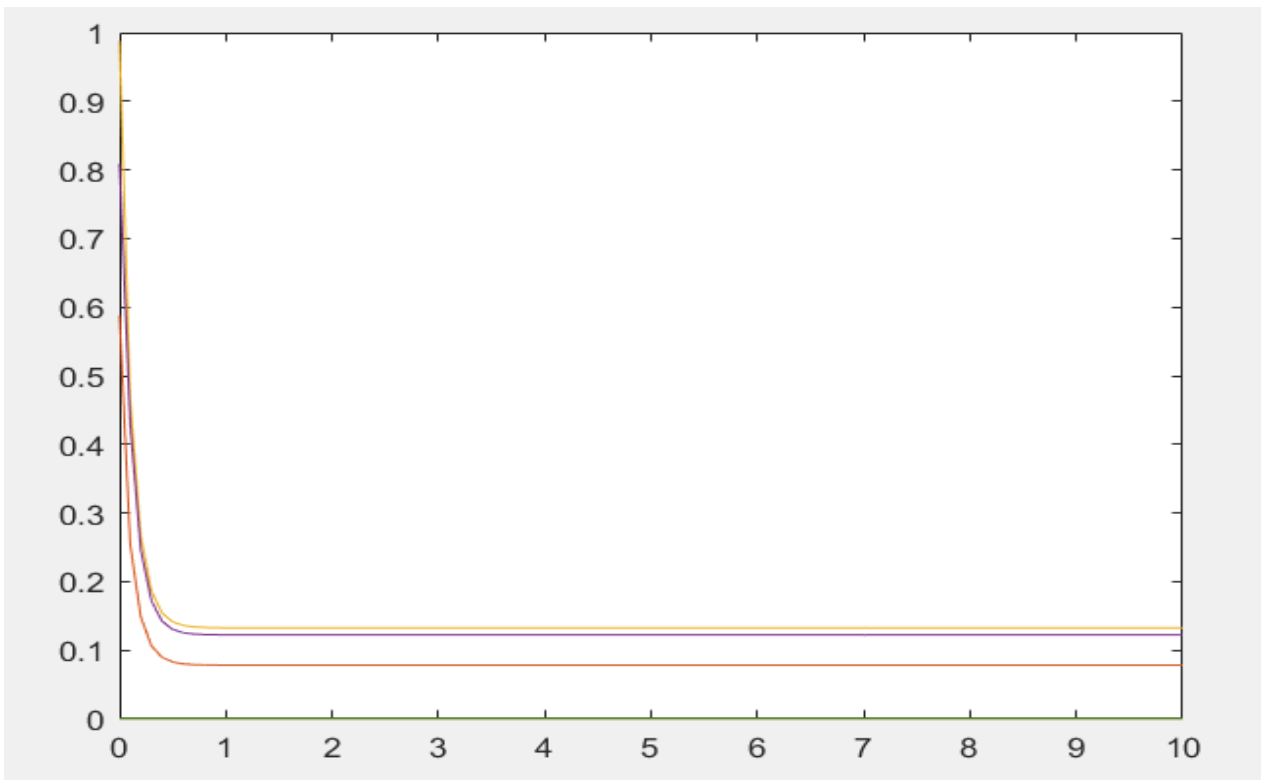


FIGURE 2(a): Temperature distribution in time for $a = -1$

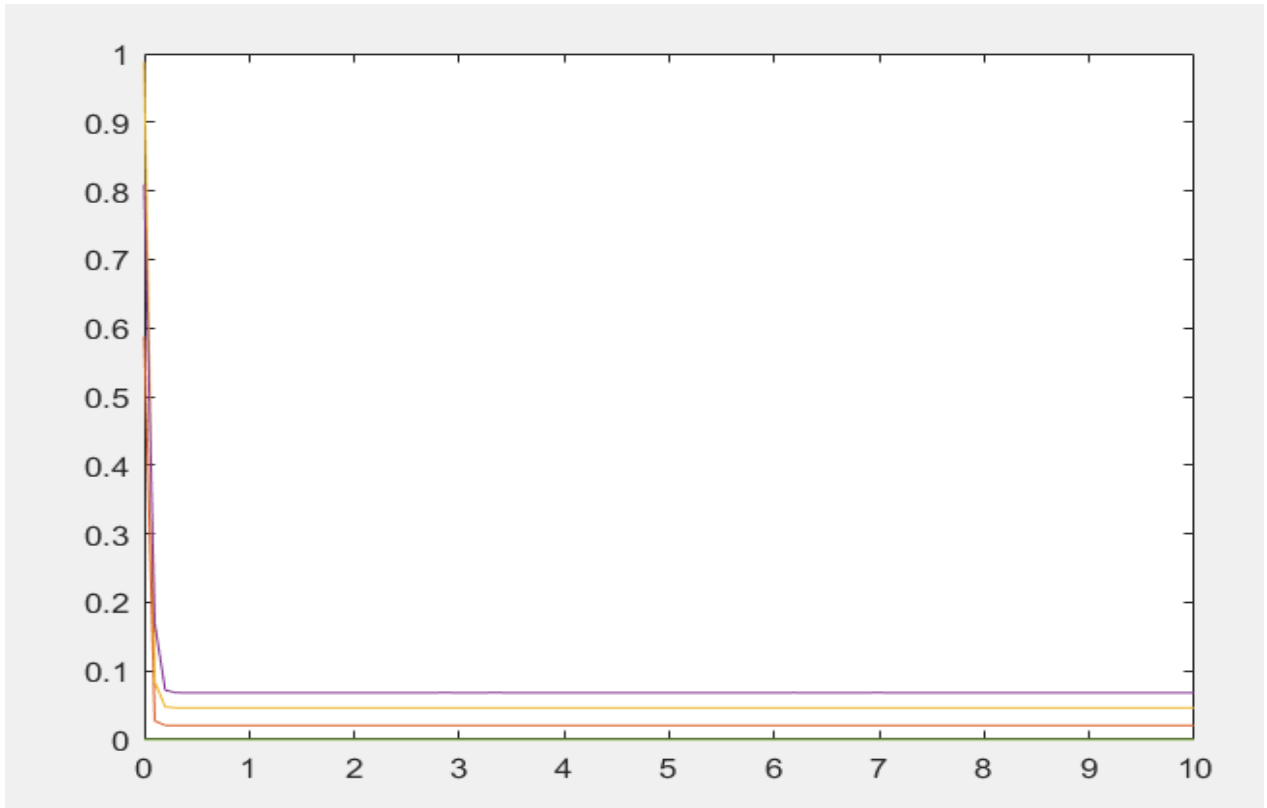


FIGURE 2(b): Temperature distribution in time for $a = -10$

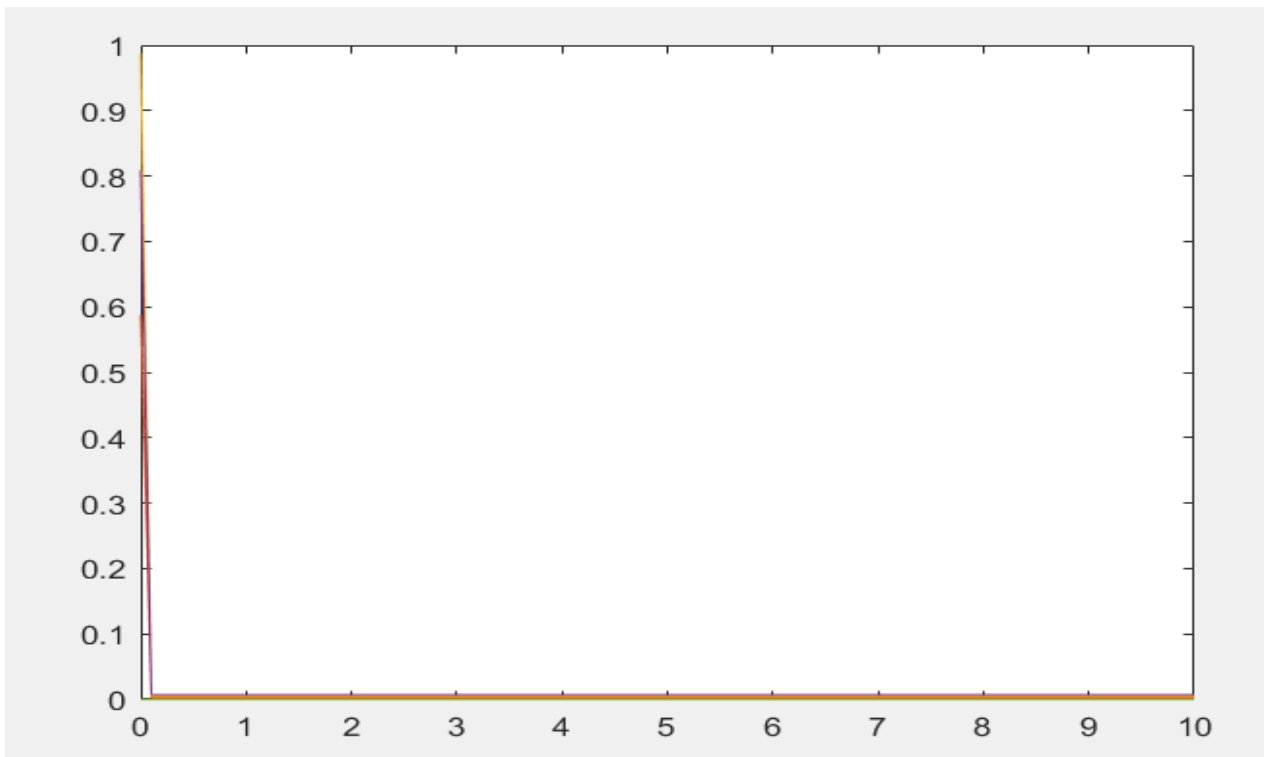


FIGURE 2(c): Temperature distribution in time for $a = -100$

IV. OBSERVATION AND CONCLUSIONS

For the full order solution of the problem, the temperature at the end of time didn't go down to zero, which is because of the extra constant forcing term in our equation. Because of that even at every point of our time period it was showing a quadratic plot. Which looks like the following for different values of t .

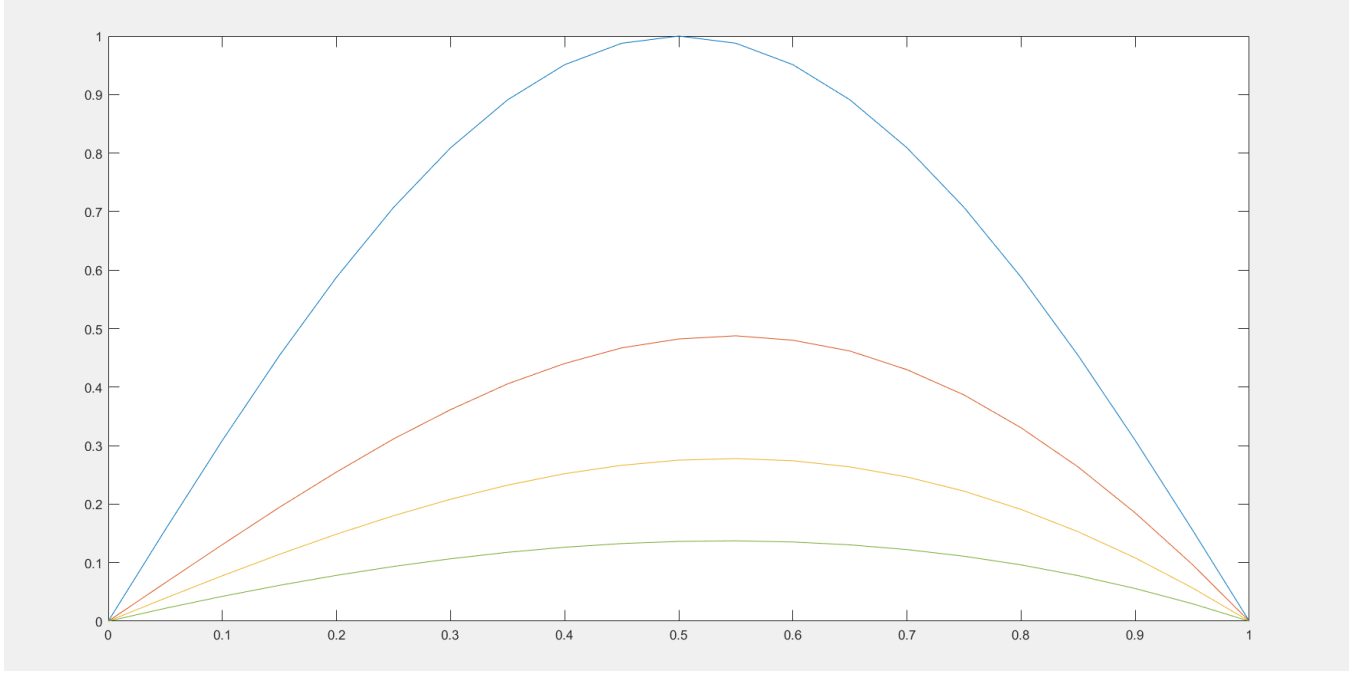


FIGURE 3: Temperature distribution in different time for $a = -1$

In the POD mode, we calculated the $\mathcal{E}_{POD}(K)$ for $K = 1, 2, 3, 4$ and found out the following result:

K	$\mathcal{E}_{POD}(K)$
1	2.39×10^{-5}
2	7.24×10^{-13}
3	1.19×10^{-18}
4	8.21×10^{-22}

So, it is evident that although we could not show the final values of temperature distribution from the reduced model, by calculating with the eigenvalues numerically, we found the $\mathcal{E}_{POD}(K)$ to be consistent with our target.

REFERENCES

1. Chatterjee, A. (2000). An introduction to the proper orthogonal decomposition. Current Science, 78(7), 808-817.
2. www.luminohope.org
3. www.wikipedia.org

APPENDIX

The MATLAB code

```
clc
clear all

a=-1; %3 different values
N=20; %number of spatial subdivisions
h=1/N; %del_x
n=100; %%number of temporal subdivisions
x=linspace(0,1,N+1);
x0=sin(pi*x); %initial condition
dt=10/n;
t=linspace(0,10,n+1);

%Matrices initialization:
Mh=sparse(N+1,N+1);
Kh=sparse(N+1,N+1);
Fh=sparse(N+1,N+1);
Nh=sparse(N+1,N+1);
b=sparse(N+1,1);
phi=sparse(N+1,N+1);

%Inputting matrices elements:

for i=1:N+1
    phi(i,i)=1;
    for j=2:N
        if abs(i-j)==1
            Kh(i,j)=-1/h;
            Mh(i,j)=h/6;
            if i>j
                Fh(i,j)=-1/2;
            else
                Fh(i,j)=1/2;
            end
        elseif i==j
            Kh(i,j)=2/h;
            Mh(i,j)=2*h/3;
            Fh(i,j)=0;
        end
    end
end
end
Kh(1,1)=1/h;
Mh(1,1)=h/3;
Kh(N+1,N+1)=1/h;
```

```

Mh(N+1,N+1)=h/3;
Fh(1,1)=-1/2;
Fh(N+1,N+1)=1/2;
b(:)=h;
b(1)=h/2;
b(N+1)=h/2;
Nh=Mh;

%omitting the boundary nodes:
phi(1,:)=[];
phi(N,:)=[];
phi(:,1)=[];
phi(:,N)=[];

Mh(1,:)=[];
Mh(N,:)=[];
Mh(:,1)=[];
Mh(:,N)=[];

Kh(1,:)=[];
Kh(N,:)=[];
Kh(:,1)=[];
Kh(:,N)=[];

Fh(1,:)=[];
Fh(N,:)=[];
Fh(:,1)=[];
Fh(:,N)=[];

Nh(1,:)=[];
Nh(N,:)=[];
Nh(:,1)=[];
Nh(:,N)=[];

b(1)=[];
b(N)=[];
x0(1)=[];
x0(N)=[];

%unified matrix:
RHS=-Kh+a*Fh+Nh;

%forming a y'=[A]y+[b] equation:
RHS1=inv(Mh)*RHS;
b1=inv(Mh)*b;

%solving the system with ode45:

```

```

[t,yp]=ode45(@ (t,yp) RHS1*yp+b1,t,x0);

yfinal=sparse(101,21);
yfinal(:,2:N)=yp;
plot(x,yfinal(1,:));
hold on;
plot(x,yfinal(2,:));
hold on;
plot(x,yfinal(3,:));
hold on;
plot(x,yfinal(80,:));
hold on;
plot(x,yfinal(101,:));
hold off

%surf(t,x,transpose(yfinal));

%.....POD PART.....%
K=1; % this value is changed manually
L=yp*phi*transpose(yp);
[vec,val]=eig(L);
phistar=sparse(K+1,1);
for i=1:K+1
    for j=1:n+1

phistar(i)=phistar(i)+(1/sqrt(val(i,i)))*vec(j,i)*yfinal(j,i);
        end
    end
ypod=sparse(101*21);
%for j=1:n+1
%    ypod(j,i)=
top=0;bottom=0;
for i=K+2:n+1
    top=top+val(i,i);
end
for i=1:n+1
    bottom=bottom+val(i,i);
end
epod=abs(top/bottom)

```